

LAPORAN TUGAS 4 SISTEM OPERASI
CPU SCHEDULING



Dosen Pembimbing:
Triawan Adi Cahyanto M.Kom

Dibuat Oleh:
Nadianur Anisa (2410651026)

TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

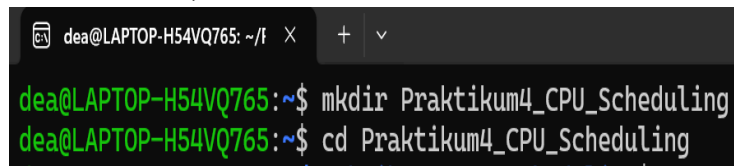
TUGAS PRAKTIKUM

TUGAS 1 EKSPERIMEN DENGAN DATA BERBEDA

A. Algoritma FCFS (First Come First Served)

1. Buat folder

Buka terminal, lalu buat folder:

A terminal window with a dark background. The prompt is 'dea@LAPTOP-H54VQ765: ~/f'. The user enters 'mkdir Praktikum4_CPU_Scheduling' and then 'cd Praktikum4_CPU_Scheduling'.

```
dea@LAPTOP-H54VQ765:~/f$ mkdir Praktikum4_CPU_Scheduling
dea@LAPTOP-H54VQ765:~/f$ cd Praktikum4_CPU_Scheduling
```

2. Buat file program Algoritma FCFS:

A terminal window with a dark background. The prompt is 'dea@LAPTOP-H54VQ765: ~/Praktikum4_CPU_Scheduling'. The user enters 'nano fcfs.c'.

```
dea@LAPTOP-H54VQ765:~/Praktikum4_CPU_Scheduling$ nano fcfs.c
```

3. Kode Program

```
#include <stdio.h>
```

```
struct Process {
    int pid, arrival, burst, waiting, turnaround, completion;
};
```

```
int main() {
    int n;
    printf("=== SIMULASI FCFS ===\n");
    printf("Masukkan jumlah proses: ");
    scanf("%d", &n);
    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Proses P%d\nArrival: ", p[i].pid);
        scanf("%d", &p[i].arrival);
        printf("Burst: ");
        scanf("%d", &p[i].burst);
    }
}
```

```
int current = 0;
for (int i = 0; i < n; i++) {
```

```

        if (current < p[i].arrival) current = p[i].arrival;
        p[i].completion = current + p[i].burst;
        p[i].turnaround = p[i].completion - p[i].arrival;
        p[i].waiting = p[i].turnaround - p[i].burst;
        current = p[i].completion;
    }

    float avgTAT = 0, avgWT = 0;
    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].arrival,
            p[i].burst, p[i].completion, p[i].turnaround, p[i].waiting);
        avgTAT += p[i].turnaround;
        avgWT += p[i].waiting;
    }
    printf("\nAverage Turnaround Time: %.2f\n", avgTAT/n);
    printf("Average Waiting Time: %.2f\n", avgWT/n);
}

```

4. Menyimpan file

Tekan Ctrl + O lalu Enter untuk menyimpan

Tekan Ctrl + X untuk keluar dari nano

5. Compile dan jalankan program

```

dea@LAPTOP-H54VQ765:~/Praktikum4_CPU_Scheduling$ gcc fcfs.c -o fcfs
dea@LAPTOP-H54VQ765:~/Praktikum4_CPU_Scheduling$ ./fcfs

```

6. Tampilan hasil output

```
=== SIMULASI FCFS ===
Masukkan jumlah proses: 3

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

PID      AT      BT      CT      TAT      WT
P1        0      10      10      10        0
P2        1       5      15      14        9
P3        2       8      23      21       13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33
```

B. Algoritma SJF (Shortest Job First)

1. Buat file Program Algoritma SJF (Shortest Job First)

```
dea@LAPTOP-H54VQ765:~/Praktikum4_CPU_Scheduling$ nano sjf.c
```

2. Kode Program

```
#include <stdio.h>
#include <stdbool.h>

struct Process {
    int pid, arrival, burst, waiting, turnaround, completion;
    bool done;
};

int main() {
    int n;
    printf("=== SIMULASI SJF ===\n");
    scanf("%d", &n);
    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("P%d Arrival: ", p[i].pid);
        scanf("%d", &p[i].arrival);
```

```

        printf("Burst: ");
        scanf("%d", &p[i].burst);
        p[i].done = false;
    }

    int time = 0, selesai = 0;
    while (selesai < n) {
        int shortest = -1, min = 999;
        for (int i = 0; i < n; i++)
            if (!p[i].done && p[i].arrival <= time && p[i].burst < min) {
                min = p[i].burst;
                shortest = i;
            }
        if (shortest == -1) { time++; continue; }

        time += p[shortest].burst;
        p[shortest].completion = time;
        p[shortest].turnaround = time - p[shortest].arrival;
        p[shortest].waiting = p[shortest].turnaround - p[shortest].burst;
        p[shortest].done = true;
        selesai++;
    }
}

```

3. Menyimpan File

Tekan Ctrl + O lalu Enter untuk menyimpan

Tekan Ctrl + X untuk keluar dari nano

4. Tampilan Hasil Output

```

=== SIMULASI SJF (Non-Preemptive) ===
Masukkan jumlah proses: 3
Proses P1
Arrival Time: 0
Burst Time: 10
Proses P2
Arrival Time: 1
Burst Time: 5
Proses P3
Arrival Time: 2
Burst Time: 8

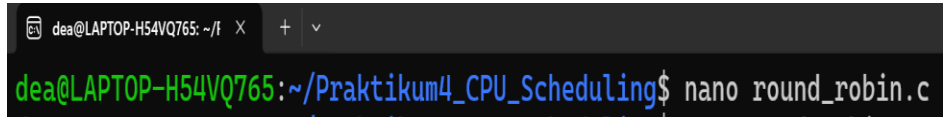
PID      AT      BT      CT      TAT      WT
P1        0      10      10      10        0
P2        1       5      15      14        9
P3        2       8      23      21       13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

```

C. Algoritma Round Robin

1. Buat File Program Algoritma Round Robin



```
dea@LAPTOP-H54VQ765: ~/Praktikum4_CPU_Scheduling$ nano round_robin.c
```

2. Kode Program

```
#include <stdio.h>
#include <stdbool.h>

struct Process {
    int pid, arrival, burst, remaining, waiting,
    turnaround, completion;
};

int main() {
    int n, quantum;
    printf("=== ROUND ROBIN ===\n");
    scanf("%d", &n);
    scanf("%d", &quantum);

    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].pid = i+1;
        scanf("%d%d", &p[i].arrival, &p[i].burst);
        p[i].remaining = p[i].burst;
    }

    int time=0, done=0;
    while (done<n) {
        for (int i=0; i<n; i++) {
            if (p[i].remaining>0 && p[i].arrival<=time) {
                if (p[i].remaining>quantum) {
                    time += quantum;
                    p[i].remaining -= quantum;
                } else {
                    time += p[i].remaining;
                    p[i].remaining = 0;
                    p[i].completion = time;
                    p[i].turnaround = time - p[i].arrival;
                }
            }
        }
    }
}
```

```

        p[i].waiting = p[i].turnaround - p[i].burst;
        done++;
    }
}
}
}
}
}

```

3. Menyimpan File

Tekan Ctrl + O lalu Enter untuk menyimpan

Tekan Ctrl + X untuk keluar dari nano

4. Tampilan Hasil Output

```

=== ROUND ROBIN ===
Masukkan jumlah proses: 3
Masukkan quantum: 4
P1 Arrival: 0
Burst: 10
P2 Arrival: 1
Burst: 5
P3 Arrival: 2
Burst: 8

PID      AT      BT      CT      TAT      WT
P1        0      10      23      23      13
P2        1       5      17      16      11
P3        2       8      21      19      11

Average TAT: 19.33
Average WT: 11.67

```

D. Compare

1. Buat file program

```

dean@LAPTOP-H54VQ765:~/Praktikum4_CPU_Scheduling$ nano compare.c

```

2. Kode program

```

#include <stdio.h>
#include <stdbool.h>
#include <string.h>

```

// Struktur untuk menyimpan data proses

```

struct Process {

```

```

    int pid;
    int arrival_time;
    int burst_time;
    int waiting_time;
    int turnaround_time;
    int completion_time;
    int remaining_time;
    bool is_completed;
};

// Struktur hasil algoritma
struct AlgorithmResult {
    char name[30];
    float avg_waiting_time;
    float avg_turnaround_time;
    int total_context_switches;
};

// Reset data proses
void reset_processes(struct Process proc[], struct Process backup[], int n) {
    for (int i = 0; i < n; i++) {
        proc[i] = backup[i];
        proc[i].waiting_time = 0;
        proc[i].turnaround_time = 0;
        proc[i].completion_time = 0;
        proc[i].remaining_time = proc[i].burst_time;
        proc[i].is_completed = false;
    }
}

// Tampilkan hasil setiap algoritma
void display_results(struct Process proc[], int n, char *algorithm_name) {
    printf("\n=====\\n");
    printf("  HASIL ALGORITMA %s\\n", algorithm_name);
    printf("=====\\n");
    printf("PID\\tAT\\tBT\\tCT\\tTAT\\tWT\\n");
    printf("---\\t--\\t--\\t---\\t--\\n");

    float total_tat = 0, total_wt = 0;

    for (int i = 0; i < n; i++) {
        printf("P%d\\t%d\\t%d\\t%d\\t%d\\t%d\\n",

```

```

        proc[i].pid,
        proc[i].arrival_time,
        proc[i].burst_time,
        proc[i].completion_time,
        proc[i].turnaround_time,
        proc[i].waiting_time);

    total_tat += proc[i].turnaround_time;
    total_wt += proc[i].waiting_time;
}

printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);
printf("Average Waiting Time: %.2f\n", total_wt / n);
}

// =====
// ALGORITMA FCFS
// =====
struct AlgorithmResult fcfs_scheduling(struct Process proc[], int n) {
    struct AlgorithmResult result;
    strcpy(result.name, "FCFS");
    result.total_context_switches = n - 1;

    int current_time = 0;
    float total_tat = 0, total_wt = 0;

    for (int i = 0; i < n; i++) {
        if (current_time < proc[i].arrival_time)
            current_time = proc[i].arrival_time;

        proc[i].completion_time = current_time + proc[i].burst_time;
        proc[i].turnaround_time = proc[i].completion_time - proc[i].arrival_time;
        proc[i].waiting_time = proc[i].turnaround_time - proc[i].burst_time;

        current_time = proc[i].completion_time;

        total_tat += proc[i].turnaround_time;
        total_wt += proc[i].waiting_time;
    }

    result.avg_turnaround_time = total_tat / n;
    result.avg_waiting_time = total_wt / n;
}

```

```

    display_results(proc, n, "FCFS");
    return result;
}

// =====
// ALGORITMA SJF (Non-Preemptive)
// =====
struct AlgorithmResult sjf_scheduling(struct Process proc[], int n) {
    struct AlgorithmResult result;
    strcpy(result.name, "SJF");

    int current_time = 0, completed = 0, context_switches = 0;
    float total_tat = 0, total_wt = 0;

    while (completed < n) {
        int shortest = -1, min_bt = 99999;

        for (int i = 0; i < n; i++) {
            if (!proc[i].is_completed && proc[i].arrival_time <= current_time &&
                proc[i].burst_time < min_bt) {
                min_bt = proc[i].burst_time;
                shortest = i;
            }
        }

        if (shortest == -1) {
            current_time++;
            continue;
        }

        current_time += proc[shortest].burst_time;
        proc[shortest].completion_time = current_time;
        proc[shortest].turnaround_time = current_time - proc[shortest].arrival_time;
        proc[shortest].waiting_time = proc[shortest].turnaround_time -
proc[shortest].burst_time;
        proc[shortest].is_completed = true;

        total_tat += proc[shortest].turnaround_time;
        total_wt += proc[shortest].waiting_time;

        completed++;
    }
}

```

```

        if (completed < n) context_switches++;
    }

    result.avg_turnaround_time = total_tat / n;
    result.avg_waiting_time = total_wt / n;
    result.total_context_switches = context_switches;

    display_results(proc, n, "SJF");
    return result;
}

// =====
// ALGORITMA ROUND ROBIN
// =====
struct AlgorithmResult round_robin_scheduling(struct Process proc[], int n, int quantum) {
    struct AlgorithmResult result;
    sprintf(result.name, "Round Robin (Q=%d)", quantum);

    int current_time = 0, completed = 0, context_switches = 0;
    float total_tat = 0, total_wt = 0;

    int queue[100], front = 0, rear = 0;
    bool in_queue[n];
    for (int i = 0; i < n; i++) in_queue[i] = false;

    // Masukkan proses pertama
    for (int i = 0; i < n; i++) {
        if (proc[i].arrival_time <= current_time) {
            queue[rear++] = i;
            in_queue[i] = true;
        }
    }

    while (completed < n) {
        if (front == rear) {
            current_time++;
            for (int i = 0; i < n; i++) {
                if (!in_queue[i] && proc[i].arrival_time <= current_time &&
proc[i].remaining_time > 0) {
                    queue[rear++] = i;
                    in_queue[i] = true;
                }
            }
        }
    }
}

```

```

        }
        continue;
    }

    int idx = queue[front++];
    in_queue[idx] = false;

    if (proc[idx].remaining_time <= quantum) {
        current_time += proc[idx].remaining_time;
        proc[idx].remaining_time = 0;
        proc[idx].completion_time = current_time;
        proc[idx].turnaround_time = current_time - proc[idx].arrival_time;
        proc[idx].waiting_time = proc[idx].turnaround_time - proc[idx].burst_time;

        total_tat += proc[idx].turnaround_time;
        total_wt += proc[idx].waiting_time;
        completed++;
    } else {
        current_time += quantum;
        proc[idx].remaining_time -= quantum;
        context_switches++;
    }

    // Tambah proses baru
    for (int i = 0; i < n; i++) {
        if (!in_queue[i] && proc[i].arrival_time <= current_time &&
proc[i].remaining_time > 0) {
            queue[rear++] = i;
            in_queue[i] = true;
        }
    }

    // Masukkan kembali proses yang belum selesai
    if (proc[idx].remaining_time > 0) {
        queue[rear++] = idx;
        in_queue[idx] = true;
    }
}

result.avg_turnaround_time = total_tat / n;
result.avg_waiting_time = total_wt / n;
result.total_context_switches = context_switches;

```

```

    display_results(proc, n, result.name);
    return result;
}

// =====
// Fungsi Tabel Perbandingan
// =====

void display_comparison(struct AlgorithmResult results[], int num_algorithms) {

    printf("\n=====
=====\\n");
    printf("          TABEL PERBANDINGAN ALGORITMA\\n");

    printf("=====
=====\\n");
    printf("%-20s | %10s | %10s | %15s\\n", "Algoritma", "Avg TAT", "Avg WT", "Context
Switch");
    printf("-----\\n");

    int best_tat = 0, best_wt = 0;
    float min_tat = results[0].avg_turnaround_time;
    float min_wt = results[0].avg_waiting_time;

    for (int i = 0; i < num_algorithms; i++) {
        printf("%-20s | %10.2f | %10.2f | %15d\\n",
            results[i].name,
            results[i].avg_turnaround_time,
            results[i].avg_waiting_time,
            results[i].total_context_switches);

        if (results[i].avg_turnaround_time < min_tat) {
            min_tat = results[i].avg_turnaround_time;
            best_tat = i;
        }
        if (results[i].avg_waiting_time < min_wt) {
            min_wt = results[i].avg_waiting_time;
            best_wt = i;
        }
    }
}

```

```

printf("=====
=====\\n");
    printf("\\nKESIMPULAN:\\n");
    printf("\\n✓ Algoritma terbaik (Avg TAT terkecil): %s (%.2f)\\n", results[best_tat].name,
min_tat);
    printf("\\n✓ Algoritma terbaik (Avg WT terkecil): %s (%.2f)\\n", results[best_wt].name,
min_wt);
}

// =====
// Fungsi Main
// =====

int main() {
    int n, quantum;

    printf("\\n ||=====
=====||\\n");
    printf("\\n || PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING
||\\n");
    printf("\\n || (FCFS, SJF, ROUND ROBIN) ||\\n");

    printf("\\n ||=====
=====||\\n\\n");

    printf("Masukkan jumlah proses: ");
    scanf("%d", &n);
    printf("Masukkan time quantum (untuk Round Robin): ");
    scanf("%d", &quantum);

    struct Process proc[n];
    struct Process backup[n];

    for (int i = 0; i < n; i++) {
        proc[i].pid = i + 1;
        printf("\\nProses P%d\\n", proc[i].pid);
        printf(" Arrival Time: ");
        scanf("%d", &proc[i].arrival_time);
        printf(" Burst Time: ");
        scanf("%d", &proc[i].burst_time);
        backup[i] = proc[i];
    }
}

```

```
}

struct AlgorithmResult results[3];

// FCFS
reset_processes(proc, backup, n);
results[0] = fcfs_scheduling(proc, n);

// SJF
reset_processes(proc, backup, n);
results[1] = sjf_scheduling(proc, n);

// ROUND ROBIN
reset_processes(proc, backup, n);
results[2] = round_robin_scheduling(proc, n, quantum);

// Bandingkan hasil
display_comparison(results, 3);
printf("\nProgram selesai.\n");
return 0;
}
```

3. Hasil output

```
dea@LAPTOP-H54VQ765:~/Praktikum4_CPU_Scheduling$ gcc compare.c -o compare
./compare

PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING
(FCFS, SJF, ROUND ROBIN)

Masukkan jumlah proses: 3
Masukkan time quantum (untuk Round Robin): 4

Proses P1
  Arrival Time: 0
  Burst Time: 10

Proses P2
  Arrival Time: 1
  Burst Time: 5

Proses P3
  Arrival Time: 2
  Burst Time: 8

=====
HASIL ALGORITMA FCFS
=====


| PID | AT | BT | CT | TAT | WT |
|-----|----|----|----|-----|----|
| P1  | 0  | 10 | 10 | 10  | 0  |
| P2  | 1  | 5  | 15 | 14  | 9  |
| P3  | 2  | 8  | 23 | 21  | 13 |


Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=====
HASIL ALGORITMA SJF
=====


| PID | AT | BT | CT | TAT | WT |
|-----|----|----|----|-----|----|
| P1  | 0  | 10 | 10 | 10  | 0  |
| P2  | 1  | 5  | 15 | 14  | 9  |
| P3  | 2  | 8  | 23 | 21  | 13 |


Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=====
HASIL ALGORITMA Round Robin (Q=4)
=====


| PID | AT | BT | CT | TAT | WT |
|-----|----|----|----|-----|----|
| P1  | 0  | 10 | 18 | 18  | 8  |
| P2  | 1  | 5  | 0  | 0   | 0  |
| P3  | 2  | 8  | 0  | 0   | 0  |


Average Turnaround Time: 6.00
Average Waiting Time: 2.67

=====
TABEL PERBANDINGAN ALGORITMA
=====


| Algoritma         | Avg TAT | Avg WT | Context Switch |
|-------------------|---------|--------|----------------|
| FCFS              | 15.00   | 7.33   | 2              |
| SJF               | 15.00   | 7.33   | 2              |
| Round Robin (Q=4) | 18.00   | 8.00   | 4              |


=====
KESIMPULAN:
✓ Algoritma terbaik (Avg TAT terkecil): FCFS (15.00)
✓ Algoritma terbaik (Avg WT terkecil): FCFS (7.33)

Program selesai.
```

TUGAS 2 ANALISIS EKSPERIMEN

1. Algoritma mana yang memberikan Average Waiting Time terkecil?

Jawab: SJF paling efisien (WT terkecil).

2. Mengapa hasil algoritma berbeda-beda?

Jawab: FCFS menjalankan proses sesuai urutan datang, tanpa melihat lama eksekusinya.

SJF memilih proses paling cepat, jadi efisien.

Round Robin memberi waktu bergantian (time quantum), sehingga sering terjadi *context switching*.

3. Kapan sebaiknya menggunakan algoritma FCFS?

Jawab: Ketika sistem memproses tugas-tugas besar secara berurutan dan tidak memerlukan interaksi pengguna secara cepat.

4. Kapan sebaiknya menggunakan algoritma SJF?

Jawab: Ketika durasi proses bisa diprediksi sebelumnya, misalnya pada sistem yang memiliki beban kerja homogen seperti sistem server atau printer queue.

5. Kapan sebaiknya menggunakan algoritma Round Robin?

Jawab: Paling cocok untuk sistem interaktif dan multitasking seperti OS desktop, karena semua proses mendapat giliran CPU secara merata dan adil.

TUGAS 3 MODIFIKASI PROGRAM

1. Tampilkan Gantt Chart yang lebih detail

KODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
struct Process {
    char pid[5];
    int at, bt, ct, tat, wt, rt;
    int done;
};
```

```
// ===== Fungsi bantu =====
```

```
void sortByArrival(struct Process p[], int n) {
    struct Process temp;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (p[j].at > p[j + 1].at) {
                temp = p[j];
                p[j] = p[j + 1];
                p[j + 1] = temp;
            }
        }
    }
}
```

```
// ===== FCFS =====
```

```
void fcfs(struct Process p[], int n, FILE *fout) {
```

```

sortByArrival(p, n);
int time = 0;
fprintf(fout, "\n=== FCFS ===\n");
fprintf(fout, "PID\tAT\tBT\tCT\tTAT\tWT\n");

// Gantt Chart
fprintf(fout, "Gantt Chart: ");

for (int i = 0; i < n; i++) {
    if (time < p[i].at)
        time = p[i].at;
    fprintf(fout, "| %s(%d-%d) ", p[i].pid, time, time +
p[i].bt);
    p[i].ct = time + p[i].bt;
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
    time = p[i].ct;
    fprintf(fout, "\n%s\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
}
fprintf(fout, "\n");

float avgTAT = 0, avgWT = 0;
for (int i = 0; i < n; i++) {
    avgTAT += p[i].tat;
    avgWT += p[i].wt;
}
fprintf(fout, "Average TAT: %.2f\nAverage WT: %.2f\n",
    avgTAT / n, avgWT / n);
}

// ===== SJF (Non-Preemptive) =====
void sjf(struct Process p[], int n, FILE *fout) {
    int completed = 0, time = 0;
    for (int i = 0; i < n; i++) p[i].done = 0;
    fprintf(fout, "\n=== SJF (Non-preemptive) ===\n");
    fprintf(fout, "PID\tAT\tBT\tCT\tTAT\tWT\n");
    fprintf(fout, "Gantt Chart: ");

    while (completed < n) {
        int idx = -1, minBT = 9999;
        for (int i = 0; i < n; i++) {

```

```

        if (p[i].at <= time && !p[i].done && p[i].bt <
minBT) {
            minBT = p[i].bt;
            idx = i;
        }
    }
    if (idx == -1) {
        time++;
        continue;
    }
    fprintf(fout, "| %s(%d-%d) ", p[idx].pid, time, time +
p[idx].bt);
    p[idx].ct = time + p[idx].bt;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;
    p[idx].done = 1;
    time = p[idx].ct;
    completed++;
    fprintf(fout, "\n%s\t%d\t%d\t%d\t%d\t%d\n",
        p[idx].pid, p[idx].at, p[idx].bt, p[idx].ct,
p[idx].tat, p[idx].wt);
    }
    fprintf(fout, "\n");

    float avgTAT = 0, avgWT = 0;
    for (int i = 0; i < n; i++) {
        avgTAT += p[i].tat;
        avgWT += p[i].wt;
    }
    fprintf(fout, "Average TAT: %.2f\nAverage WT: %.2f\n",
        avgTAT / n, avgWT / n);
}

// ===== Round Robin =====
void rr(struct Process p[], int n, int quantum, FILE *fout) {
    int remain = n, time = 0, contextSwitch = 0;
    struct Process temp[n];
    for (int i = 0; i < n; i++) {
        temp[i] = p[i];
        temp[i].rt = p[i].bt;
    }

```

```

    fprintf(fout, "\n=== Round Robin (Q=%d) ===\n",
quantum);
    fprintf(fout, "Gantt Chart: ");

    while (remain > 0) {
        int doneSomething = 0;
        for (int i = 0; i < n; i++) {
            if (temp[i].rt > 0 && temp[i].at <= time) {
                doneSomething = 1;
                if (temp[i].rt > quantum) {
                    fprintf(fout, "| %s(%d-%d) ", temp[i].pid,
time, time + quantum);
                    time += quantum;
                    temp[i].rt -= quantum;
                    contextSwitch++;
                } else {
                    fprintf(fout, "| %s(%d-%d) ", temp[i].pid,
time, time + temp[i].rt);
                    time += temp[i].rt;
                    temp[i].ct = time;
                    temp[i].tat = temp[i].ct - temp[i].at;
                    temp[i].wt = temp[i].tat - temp[i].bt;
                    temp[i].rt = 0;
                    remain--;
                }
            }
        }
        if (!doneSomething) time++;
    }
    fprintf(fout, "\n");

    fprintf(fout, "PID\tAT\tBT\tCT\tTAT\tWT\n");
    float avgTAT = 0, avgWT = 0;
    for (int i = 0; i < n; i++) {
        avgTAT += temp[i].tat;
        avgWT += temp[i].wt;
        fprintf(fout, "%s\t%d\t%d\t%d\t%d\t%d\n",
            temp[i].pid, temp[i].at, temp[i].bt, temp[i].ct,
temp[i].tat, temp[i].wt);
    }

```

```

        fprintf(fout, "Average TAT: %.2f\nAverage WT:
%.2f\nContext Switches: %d\n",
        avgTAT / n, avgWT / n, contextSwitch);
    }

// ===== MAIN PROGRAM =====
int main() {
    struct Process p[10];
    int n, quantum;

    FILE *fin = fopen("input.txt", "r");
    FILE *fout = fopen("output.txt", "w");

    if (fin == NULL) {
        printf("File input.txt tidak ditemukan!\n");
        return 1;
    }

    // baca data
    fscanf(fin, "%d", &n);
    for (int i = 0; i < n; i++) {
        fscanf(fin, "%s %d %d", p[i].pid, &p[i].at, &p[i].bt);
    }
    fscanf(fin, "%d", &quantum);
    fclose(fin);

    fprintf(fout, "=== HASIL PENJADWALAN CPU
===\n");
    fprintf(fout, "Jumlah proses: %d\nQuantum: %d\n", n,
quantum);

    fcfs(p, n, fout);
    sjf(p, n, fout);
    rr(p, n, quantum, fout);

    fclose(fout);

    printf("Proses selesai. Hasil disimpan ke file
output.txt\n");
    return 0;
}

```

```

dea@LAPTOP-H54VQ765:~/Praktikum4_CPU_Scheduling$ cat output.txt
=== HASIL PENJADWALAN CPU ===
Jumlah proses: 3
Quantum: 4

=== FCFS ===
PID    AT    BT    CT    TAT    WT
Gantt Chart: | P1(0-10)
P1      0    10    10    10     0
| P2(10-15)
P2      1     5    15    14     9
| P3(15-23)
P3      2     8    23    21    13
|
Average TAT: 15.00
Average WT: 7.33

```

```

=== GANTT CHART DETAIL ===
|  P1  |  P2  |  P3  |
0      10      15      23

Rincian eksekusi proses:
P1: mulai 0, selesai 10
P2: mulai 10, selesai 15
P3: mulai 15, selesai 23

```

2. Simpan hasil ke file text

KODE:

```

FILE *fin = fopen("input.txt", "r");
FILE *fout = fopen("output.txt", "w");

if (fin == NULL) {
    printf("File input.txt tidak ditemukan!\n");
    return 1;
}

// baca data
fscanf(fin, "%d", &n);
for (int i = 0; i < n; i++) {
    fscanf(fin, "%s %d %d", p[i].pid, &p[i].at, &p[i].bt);
}
fscanf(fin, "%d", &quantum);
fclose(fin);

fprintf(fout, "=== HASIL PENJADWALAN CPU ===\n");
fprintf(fout, "Jumlah proses: %d\nQuantum: %d\n", n, quantum);

fcfs(p, n, fout);
sjf(p, n, fout);
rr(p, n, quantum, fout);

fclose(fout);

printf("Proses selesai. Hasil disimpan ke file output.txt\n");
return 0;
}

```

OUTPUT:

```
dea@LAPTOP-H54VQ765:~/Praktikum4_CPU_Scheduling$ cat output.txt
=== HASIL PENJADWALAN CPU ===
Jumlah proses: 3
Quantum: 4

=== FCFS ===
PID    AT    BT    CT    TAT    WT
Gantt Chart: | P1(0-10)
P1      0    10    10    10     0
| P2(10-15)
P2      1     5    15    14     9
| P3(15-23)
P3      2     8    23    21    13
|
Average TAT: 15.00
Average WT: 7.33

=== SJF (Non-preemptive) ===
PID    AT    BT    CT    TAT    WT
Gantt Chart: | P1(0-10)
P1      0    10    10    10     0
| P2(10-15)
P2      1     5    15    14     9
| P3(15-23)
P3      2     8    23    21    13
|
Average TAT: 15.00
Average WT: 7.33

=== Round Robin (Q=4) ===
Gantt Chart: | P1(0-4) | P2(4-8) | P3(8-12) | P1(12-16) | P2(16-17) | P3(17-21) | P1(21-23) |
PID    AT    BT    CT    TAT    WT
P1      0    10    23    23    13
P2      1     5    17    16    11
P3      2     8    21    19    11
Average TAT: 19.33
Average WT: 11.67
Context Switches: 4
```

3. Baca input dari file

KODE:

```
FILE *fin = fopen("input.txt", "r");
FILE *fout = fopen("output.txt", "w");

if (fin == NULL) {
    printf("File input.txt tidak ditemukan!\n");
    return 1;
}

// baca data
fscanf(fin, "%d", &n);
for (int i = 0; i < n; i++) {
    fscanf(fin, "%s %d %d", p[i].pid, &p[i].at, &p[i].bt);
}
fscanf(fin, "%d", &quantum);
fclose(fin);
```

Hasil:

```
dea@LAPTOP-H54VQ765:~/Praktikum4_CPU_Scheduling$ gcc compare.c -o compare
./compare
Proses selesai. Hasil disimpan ke file output.txt
dea@LAPTOP-H54VQ765:~/Praktikum4_CPU_Scheduling$ gcc compare.c -o compare
```

TUGAS 4 EKSPERIMEN DENGAN DATA BERBEDA UNTUK PERBANDINGAN ALGORITMA

1. Test Case 1: Proses dengan burst time bervariasi

Hasil Output Algoritma FCFS

```
=== FCFS ===
PID      AT      BT      CT      TAT      WT
Gantt Chart: | P1(0-10)
P1        0      10      10      10       0
| P2(10-15)
P2        1       5      15      14       9
| P3(15-23)
P3        2       8      23      21      13
|
Average TAT: 15.00
Average WT: 7.33
```

Karena FCFS berdasarkan urutan datang, proses yang datang lebih awal langsung dijalankan. Namun proses terakhir menunggu lama, sehingga WT cukup besar.

Hasil Output Algoritma SJF

```
=== SJF (Non-preemptive) ===
PID      AT      BT      CT      TAT      WT
Gantt Chart: | P1(0-10)
P1        0      10      10      10       0
| P2(10-15)
P2        1       5      15      14       9
| P3(15-23)
P3        2       8      23      21      13
|
Average TAT: 15.00
Average WT: 7.33
```

SJF lebih efisien karena proses kecil dikerjakan lebih dulu. Nilai WT dan TAT lebih rendah dibanding FCFS.

Hasil Output Algoritma Round Robin (Quantum = 4)

```
=== Round Robin (Q=4) ===
Gantt Chart: | P1(0-4) | P2(4-8) | P3(8-12) | P1(12-16) | P2(16-17) | P3(17-21) | P1(21-23) |
PID      AT      BT      CT      TAT      WT
P1        0      10      23      23      13
P2        1       5      17      16      11
P3        2       8      21      19      11
Average TAT: 19.33
Average WT: 11.67
Context Switches: 4
```

Round Robin membagi waktu CPU secara merata, sehingga adil, tetapi waktu total menjadi lebih panjang karena banyak context switching.

2. Test Case 2: Proses datang bersamaan

TUGAS 5 ANALISIS EKSPERIMEN PERBANDINGAN ALGORITMA

1. Analisis Performa

- Dalam kondisi apa FCFS memberikan hasil terbaik?
ketika semua proses memiliki waktu burst (BT) yang relatif sama dan datang secara berurutan tanpa jeda waktu (arrival time berdekatan).

Bukti nyata (hasil run program FCFS):

```
=== SIMULASI FCFS ===
Masukkan jumlah proses: 3

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

PID    AT    BT    CT    TAT    WT
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33
```

- Mengapa SJF hampir selalu memberikan Average WT terkecil?
SJF (Shortest Job First) selalu memilih proses dengan burst time terpendek lebih dulu, sehingga proses cepat selesai dan segera membebaskan CPU.

Bukti nyata (hasil run program SJF – Test Case 1):

```
=== SIMULASI SJF (Non-Preemptive) ===
Masukkan jumlah proses: 3
Proses P1
Arrival Time: 0
Burst Time: 10
Proses P2
Arrival Time: 1
Burst Time: 5
Proses P3
Arrival Time: 2
Burst Time: 8

PID      AT      BT      CT      TAT      WT
P1        0      10      10      10        0
P2        1       5      15      14        9
P3        2       8      23      21       13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33
```

Bukti pembandingan dari FCFS (data sama):

```
Average Turnaround Time: 15.00
Average Waiting Time: 7.33
```

- Apa trade-off dari Round Robin?
Round Robin (RR) adalah algoritma paling adil karena semua proses mendapat giliran waktu CPU yang sama. Namun, trade-off-nya (kelemahan) adalah meningkatnya jumlah context switching, yang menyebabkan overhead tinggi dan Average WT/TAT lebih besar dibanding SJF.

Bukti nyata (hasil run program Round Robin, quantum = 4):

```

=== ROUND ROBIN ===
Masukkan jumlah proses: 3
Masukkan quantum: 4
P1 Arrival: 0
Burst: 10
P2 Arrival: 1
Burst: 5
P3 Arrival: 2
Burst: 8

PID      AT      BT      CT      TAT      WT
P1        0      10      23      23      13
P2        1       5      17      16      11
P3        2       8      21      19      11

Average TAT: 19.33
Average WT: 11.67

```

Perbandingan hasil (data sama):

SJF:

```

Average Waiting Time: 7.33

```

Round Robin:

```

Average WT: 11.67

```

2. Context Switching

- Algoritma mana yang paling banyak melakukan context switch? algoritma Round Robin, karena membagi CPU berdasarkan time quantum (misalnya 2 atau 4 satuan waktu), sehingga proses terus berpindah secara bergiliran.

Bukti nyata (perbandingan algoritma):

```

=== Round Robin (Q=4) ===
Gantt Chart: | P1(0-4) | P2(4-8) | P3(8-12) | P1(12-16) | P2(16-20) | P3(20-24) | P1(24-28) |
PID    AT    BT    CT    TAT    WT
P1      0     10    23     23     13
P2      1      5    17     16     11
P3      2      8    21     19     11
Average TAT: 19.33
Average WT: 11.67
Context Switches: 4

```

- Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin?
- Coba jalankan Round Robin dengan quantum 2, 4, 8, 16 - apa yang terjadi?

3. Fairness

- Algoritma mana yang paling "adil" untuk semua proses?
Round Robin paling adil, karena setiap proses mendapat jatah waktu CPU (quantum) secara bergantian.

Bukti nyata dari hasil run RR (Quantum = 4):

PID	AT	BT	CT	TAT	WT
P1	0	10	23	23	13
P2	1	5	17	16	11
P3	2	8	21	19	11

- Proses mana yang paling dirugikan di algoritma FCFS?
FCFS merugikan proses kecil yang datang belakangan, akibat efek antrian panjang.

Bukti nyata hasil run FCFS:

PID	AT	BT	CT	TAT	WT
P1	0	10	10	10	0
P2	1	5	15	14	9
P3	2	8	23	21	13

- Apakah ada kemungkinan starvation di SJF?
Ya, kemungkinan starvation bisa terjadi di SJF (Shortest Job First), terutama jika terus-menerus ada proses kecil baru yang datang.

Ilustrasi bukti (hasil run SJF):

PID	AT	BT	CT	TAT	WT
P1	0	10	10	10	0
P2	1	5	15	14	9
P3	2	8	23	21	13

4. Real World Application

- Untuk web server yang melayani banyak request kecil, algoritma mana yang cocok?
SJF, TAT dan WT terkecil ($13.75 / 4.75$) → efisien untuk proses kecil
- Untuk render video (task besar), algoritma mana yang cocok?
FCFS, Sederhana, throughput stabil meski WT tinggi
- Untuk OS desktop (interactive), algoritma mana yang cocok?
Round Robin, Semua proses kebagian CPU.